Access-Review-Delegation-Tracker API

Cloud-Native REST API on Microsoft Azure Functions

Midterm Project

CS-432-A Cloud Computing

Ty Iorfino

Instructor: Christopher Rosa

Sacred Heart University

11/3/2026

GitHub Repository: https://github.com/Tvin405/Access-Review-Delegation-Tracker-API

Azure Base URL: https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-01.azurewebsites.net/api

Table of Contents

Table of Contents 2
1. Application Overview 3
1.1 Purpose 3
1.2 Access Types Tracked 3
1.3 The Access Review Resource 3
1.4 Main Features 4
1.5 Technical Approach 4
1.6 Design Decisions 5
1.7 Limitations and Challenges 5
2. API Reference 6
2.1 Base URL 6
2.2 Authentication 6
2.3 Endpoint Summary and Error Format 6
2.4 Get All Access Reviews 7
2.5 Create an Access Review 8
2.6 Update an Access Review 9
2.7 Delete an Access Review 11
2.8 Count Access Reviews (Optional Enhancement) 12
2.9 Purge All Access Reviews (Optional Enhancement) 12
3. Azure Deployment Evidence 14
3.1 Deployment Details 14
3.2 Function App Overview 15
3.3 Resource Group 15
3.4 API Key Configuration 16
3.5 Successful Request on Azure 16
3.6 Unauthorized Request on Azure 17
4. Testing Evidence 18
4.1 Test Summary 18
4.2 Test Records 19
5. Conclusion 34

1. Application Overview

1.1 Purpose

Privileged accounts are the most valuable targets for attackers, and many organizations lose track of who holds them, who approved them and whether they are still needed. The Access-Review-Delegation-Tracker API solves this by giving security teams a single cloud-hosted record of every access delegation, including the user, the role and system, the access, the approver, the justification, and the start and expiration dates. This creates accountability and an audit trail, while expiration dates make it easy to spot access that should have been removed. It also makes periodic access reviews faster, because reviewers can pull every delegation from one API, revoke or update stale grants, and show auditors evidence of the decision.

1.2 Access Types Tracked

Access Type	Definition
PIM (Privileged Identity Management)	Provides just-in-time access to privileged roles. Users are made eligible for a role and must activate it using justification and a timespan, which permission expires after that time is up.
PAM (Privileged Access Management)	Secures and monitors privileged accounts and credentials. Uses credential vaulting, session monitoring, and controlled checkouts of elevated access.
PAW (Privileged Access Workstation)	A dedicated hardware device used only for admin tasks. It isolated privilege sessions from everyday tasks, so credentials are protected from phishing and malware.
Standard	Normal, non-privileged access that users need for daily tasks. Its still tracked and reviewed so it can be audited later.

1.3 The Access Review Resource

Field	Type	Description
id	string (UUID)	Unique identifier generated by the server on record creation.
user	string	Account that holds the access, e.g. pioneer@sacredheart.edu
role	string	Role or permissions granted, e.g. Global Admin
system	string	System or app the access applies to, e.g. Entra ID
accessType	string	PIM, PAM, PAW, Standard

Field	Type	Description
assignmentType	string	How access is held, e.g. eligible, active, permanent
requiresPAW	boolean	True if the role must be used on a privileged access workstation
grantedBy	string	Person or team who approved access to role
justification	string	Reason the access was granted
startDate	string (YYYY-MM-DD)	Date the access began
expirationDate	string (YYYY-MM-DD)	Date the access should end
status	string	Status, e.g. active, pending, expired, or revoked

1.4 Main Features

- Can create an access review record and return it with a POST request.
- List all access reviews returning all access delegations with a GET request.
- Delete an access review, you can remove a single record with DELETE request.
- Can return the total number of stored access reviews using a COUNT request.
- Can delete all records using a PURGE request, requires admin key & API key.
- Update an access review, replace an existing record with new information using PUT request.

1.5 Technical Approach

The API is written in Python using the Azure Functions V2 programming model, where each endpoint is a function registered on a single FunctionApp object with an @app.route decorator that defines its route and HTTP method. There are six functions in total: get all, create, update, delete, count and purge, all served under the /api/access-reviews route, with update and delete reading the record id from the URL path (access-reviews/{id}). Purge also calls validateAdminToken. Which checks x-admin-key against the adminkey environment variable to make sure normal users cannot destroy all the records in one click of a button.

Data is stored in an in-memory Python dictionary called access_reviews, where each key is a server-generated UUID and each value is the access review record as JSON. The data is not persistent and is cleared whenever the Function App restarts or is recycled.

Authentication uses an API key sent in the x-api-key request header. Every function first calls validateToken, which compares the header against the apikey value stored as an environment variable (local.settings.json locally, and Environment variables in the Azure portal), so the key never appears in the source code. The functions use the ANONYMOUS auth level because the API performs its own key check.

Errors are handled in one place. The code defines custom exceptions (AuthError and NotFoundError), and every function is wrapped with a handle_exceptions decorator that converts them into JSON error

responses: AuthError becomes 401, NotFoundError becomes 404, invalid or missing JSON (ValueError) becomes 400, and any unexpected error is logged and returned as 500. This keeps each endpoint short and makes every error response consistent.

The project was developed in Visual Studio Code with the Azure Functions and Azure Resources extensions. Azure Functions Core Tools ran the API locally, with Azurite emulating Azure Storage. Bruno was used to send and save test requests against both the local and Azure environments, and Git and GitHub were used for version control, with local.settings.json excluded through. gitignore to keep the API key out of the repository.

1.6 Design Decisions

- **Decision:** Server generates each id with UUID, this guarantees unique IDs and prevents clients from overwriting existing records.
- **Decision:** ANONYMOUS function auth with a custom x-api-key check. This allows for one consistent auth method at every endpoint, with the key stored in an environment variable.
- **Decision:** A single handle_exceptions wrapper for errors, it removes duplicate code and keeps every error response in the same JSON format.
- **Decision:** Microsoft resource naming convention, so each resource type, environment, and region is clear from its name.
- **Decision:** PURGE request requires a separate admin key and the base API key, this limits bulk deletion to admins, so normal users can't wipe every record.

1.7 Limitations and Challenges

The biggest limitation of this API is that data is stored in memory. Because the access_reviews dictionary only lives inside the running app, every record is lost whenever the Function App restarts, is redeployed, or is recycled by Azure after sitting idle.

A second challenge appeared during Azure testing. After posting three records, one GET returned only records A and B, and the next GET returned only record C. This happened because the Flex Consumption plan ran more than one instance of the app, and each instance kept its own separate copy of the dictionary, so requests were split between instances. To make testing reliable, I set the maximum instance count to 1, set the HTTP concurrency back to system-assigned, and added one always-ready instance so the app stayed running during testing. I removed the always-ready instance afterwards to avoid ongoing cost.

Other limitations include that the API does not do any input validate to fields inside a record, so any JSON object is accepted if it is valid, and there is no endpoint to retrieving a single record by id or filter by access type. Which could lead to a clear improvement path for the future project design and implementation.

2. API Reference

2.1 Base URL

```
Local: http://localhost:7071/api

Azure: https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-01.azurewebsites.net/api
```

2.2 Authentication

Every endpoint requires an API key sent in the x-api-key request header. The key is stored as the apikey environment variable on the Function App and is never included in the source code.
x-api-key: <SECRET>

The purge endpoint (DELETE /access-reviews) also requires a separate admin key in the x-admin-key header, so only administrators can delete all records. This key is stored as the adminkey environment variable.
x-admin-key: <SECRET>

If the API key is missing or incorrect, the API returns 401 Unauthorized:
{"error": "Unauthorized: Invalid or missing API key"}

If the admin key is missing or incorrect on a purge request, the API returns 401 Unauthorized:
{"error": "Unauthorized: Admin key required for purge"}

If a key is not configured on the server, the API returns 500 Internal Server Error.

2.3 Endpoint Summary and Error Format

Method	Route	Purpose
GET	/access-reviews	Retrieve all access reviews
POST	/access-reviews	Create a new access review
PUT	/access-reviews/{id}	Update an existing access review using ID
DELETE	/access-reviews/{id}	Delete a single access review by ID
GET	/access-reviews/count	Return the total number of access reviews
DELETE	/access-reviews	Purge all access reviews

Status Code	When It Happens
400 Bad Request	The request body is missing, isn't valid JSON, or isn't a JSON object
401 Unauthorized	The x-api-key is missing or wrong, or the x-admin-key is missing or wrong in purge
404 Not Found	No access review exists with the id in the URL
500 Internal Server Error	A key isn't configured on the server, or an unexpected error occurred

2.4 Get All Access Reviews

Endpoint: `GET /access-reviews`

Description: Returns every access review currently stored, as a JSON list. Returns an empty list when there are none.

Authentication: Required, using the x-api-key header.

Headers: x-api-key

Example Request:

```
GET https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-01.azurewebsites.net/api/access-reviews

x-api-key: <YOUR_API_KEY>
```

Example Success Response:

```
200 OK
[
  {
    "user": "big.red@shu-demo.edu",
    "role": "Global Administrator",
    "system": "SHU Microsoft Entra ID",
    "accessType": "PIM",
    "assignmentType": "eligible",
    "requiresPAW": true,
    "grantedBy": "ciso@shu-demo.edu",
    "justification": "Quarterly tenant configuration review",
    "startDate": "2026-10-20",
    "expirationDate": "2026-10-27",
    "status": "active",
    "id": "3f9c1b2e-7a4d-4e8f-9b21-6c0d5e8a1f47"
  }
]
```

Status Codes:

Code	Meaning
200 OK	Returns the list of access reviews (empty list if none)
401 Unauthorized	Missing or invalid API key

Example Error Response:

```
401 Unathorized
{"error": "Unauthorized: Invalid or missing API key"}
```

2.5 Create an Access Review

Endpoint: POST `/access-reviews`

Description: Create a new access review from the JSON body. Then the server generates a unique id and returns the full record

Authentication: Required, using the x-api-key header

Headers: x-api-key, Content-Type: application/JSON

Request Body:

```
{
  "user": "big.red@shu-demo.edu",
  "role": "Global Administrator",
  "system": "SHU Microsoft Entra ID",
  "accessType": "PIM",
  "assignmentType": "eligible",
  "requiresPAW": true,
  "grantedBy": "ciso@shu-demo.edu",
  "justification": "Quarterly tenant configuration review",
  "startDate": "2026-10-20",
  "expirationDate": "2026-10-27",
  "status": "active"
}
```

Example Request:

```
POST https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-
01.azurewebsites.net/api/access-reviews

x-api-key: <YOUR_API_KEY>
Content-Type: application/json
```

Example Success Response:

```
201 created
{
  "user": "big.red@shu-demo.edu",
  "role": "Global Administrator",
  "system": "SHU Microsoft Entra ID",
  "accessType": "PIM",
  "assignmentType": "eligible",
  "requiresPAW": true,
  "grantedBy": "ciso@shu-demo.edu",
  "justification": "Quarterly tenant configuration review",
  "startDate": "2026-10-20",
  "expirationDate": "2026-10-27",
  "status": "active",
  "id": "3f9c1b2e-7a4d-4e8f-9b21-6c0d5e8a1f47"
}
```

Status Codes:

Code	Meaning
201 Created	Access review created; response includes the new id
400 Bad Request	Missing or invalid JSON body
401 Unauthorized	Missing or invalid API key

Example Error Response:

```
400 bad request
{"error": "Bad Request: Request body must be a JSON object"}
```

2.6 Update an Access Review

Endpoint: PUT `/access-reviews/{id}`

Description: Replaces an existing access review with the JSON body, example is to revoke access or extend an expiration date.

Authentication: Required, using the x-api-key header

Headers: x-api-key, Content-Type:application/JSON

Path Parameters: id: the id returned when the access review was created

Request Body:

```
{
```

```
  "user": "big.red@shu-demo.edu",
  "role": "Global Administrator",
  "system": "SHU Microsoft Entra ID",
  "accessType": "PIM",
  "assignmentType": "eligible",
  "requiresPAW": true,
  "grantedBy": "ciso@shu-demo.edu",
  "justification": "Quarterly tenant configuration review",
  "startDate": "2026-10-20",
  "expirationDate": "2026-10-27",
  "status": "revoked"
}
```


Example Request:

```
PUT https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-
01.azurewebsites.net/api/access-reviews/3f9c1b2e-7a4d-4e8f-9b21-6c0d5e8a1f47

x-api-key: <YOUR_API_KEY>
Content-Type: application/json
```

Example Success Response:


```
200 OK
{
  "user": "big.red@shu-demo.edu",
  "role": "Global Administrator",
  "system": "SHU Microsoft Entra ID",
  "accessType": "PIM",
  "assignmentType": "eligible",
  "requiresPAW": true,
  "grantedBy": "ciso@shu-demo.edu",
  "justification": "Quarterly tenant configuration review",
  "startDate": "2026-10-20",
  "expirationDate": "2026-10-27",
  "status": "revoked",
  "id": "3f9c1b2e-7a4d-4e8f-9b21-6c0d5e8a1f47"
}
```


Status Codes:

Code	Meaning
200 OK	Access review updated
400 Bad Request	Missing or invalid JSON body

Code	Meaning
401 Unauthorized	Missing or invalid API key
404 Not Found	No access review with that id

Example Error Response:

```
404 Not Found
{"error": "Access review 'fake-id' not found"}
```

2.7 Delete an Access Review

Endpoint: `DELETE /access-reviews/{id}`

Description: Permanently removes a single access review by its id

Authentication: Required, using the x-api-key header

Headers: x-api-key

Path Parameters: id: the id of the access review to delete

Example Request:

```
DELETE https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-
01.azurewebsites.net/api/access-reviews/3f9c1b2e-7a4d-4e8f-9b21-6c0d5e8a1f47
x-api-key: <YOUR_API_KEY>
```

Example Success Response:

```
200 OK
{"message": "Access review deleted"}
```

Status Codes:

Code	Meaning
200 OK	Access review deleted
401 Unauthorized	Missing or invalid API key
404 Not Found	No access review with that id

Example Error Response:

```
404 Not Found
{"error": "Access review '3f9c1b2e-7a4d-4e8f-9b21-6c0d5e8a1f47' not found"}
```

2.8 Count Access Reviews (Optional Enhancement)

Endpoint: `GET /access-reviews/count`

Description: Returns the total number of access reviews currently stored

Authentication: Required, using the x-api-key header

Headers: x-api-key

Example Request:

```
GET https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-
01.azurewebsites.net/api/access-reviews/count
x-api-key: <YOUR_API_KEY>
```

Example Success Response:

```
200 OK
{"count": 3}
```

Status Codes:

Code	Meaning
200 OK	Returns the number of stored access reviews
401 Unauthorized	Missing or invalid API key

Example Error Response:

```
401 Unauthorized
{"error": "Unauthorized: Invalid or missing API key"}
```

2.9 Purge All Access Reviews (Optional Enhancement)

Endpoint: `DELETE /access-reviews`

Description: Deletes every stored access review at once and reports how many were removed. It requires the admin key as well as the API key to execute.

Authentication: Required, using both the x-api-key and x-admin-key headers

Headers: x-api-key , x-admin-key

Example Request:

```
DELETE https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-
01.azurewebsites.net/api/access-reviews
x-api-key: <YOUR_API_KEY>
x-admin-key: <YOUR_ADMIN_KEY>
```

Example Success Response:

```
200 OK
{"message": "All access reviews deleted", "deleted": 2}
```

Status Codes:

Code	Meaning
200 OK	All access reviews deleted; response includes how many
401 Unauthorized	Missing or invalid API key

Example Error Response:

```
401 Unauthorized
{"error": "Unauthorized: Admin key required for purge"}
```

3. Azure Deployment Evidence

3.1 Deployment Details

Item	Value
Function App name	func-accessreview-dev-eus2-001
Resource group	rg-accessreview-dev-eus2-001
Region	East US 2
Hosting plan	Flex Consumption
Runtime	Python 3.14
Base URL	https://func-accessreview-dev-eus2-001-c7d0gshua7dqh2ff.eastus2-01.azurewebsites.net/api
Naming convention	Microsoft Cloud Adoption Framework resource naming

The API was deployed from Visual Studio Code using the Azure Functions extension. I signed in to Azure, then used “Create Function App in Azure (Advanced)” to create the Function App, resource group, storage account and Application Insights. Each resource was named with the Cloud Adoption Framework convention, in the East US 2 region, on the Flex Consumption plan with the Python runtime. I then used “Deploy to Function App” to upload the code, and confirmed all six functions appeared as Enabled on the Function App’s Overview page. In the Azure portal, I added the apikey and adminkey values under Settings → Environment variables, so the keys are stored securely on the server and not in the code. Finally, I limited the app to one instance and tested every endpoint against the Azure base URL using Bruno.

3.2 Function App Overview

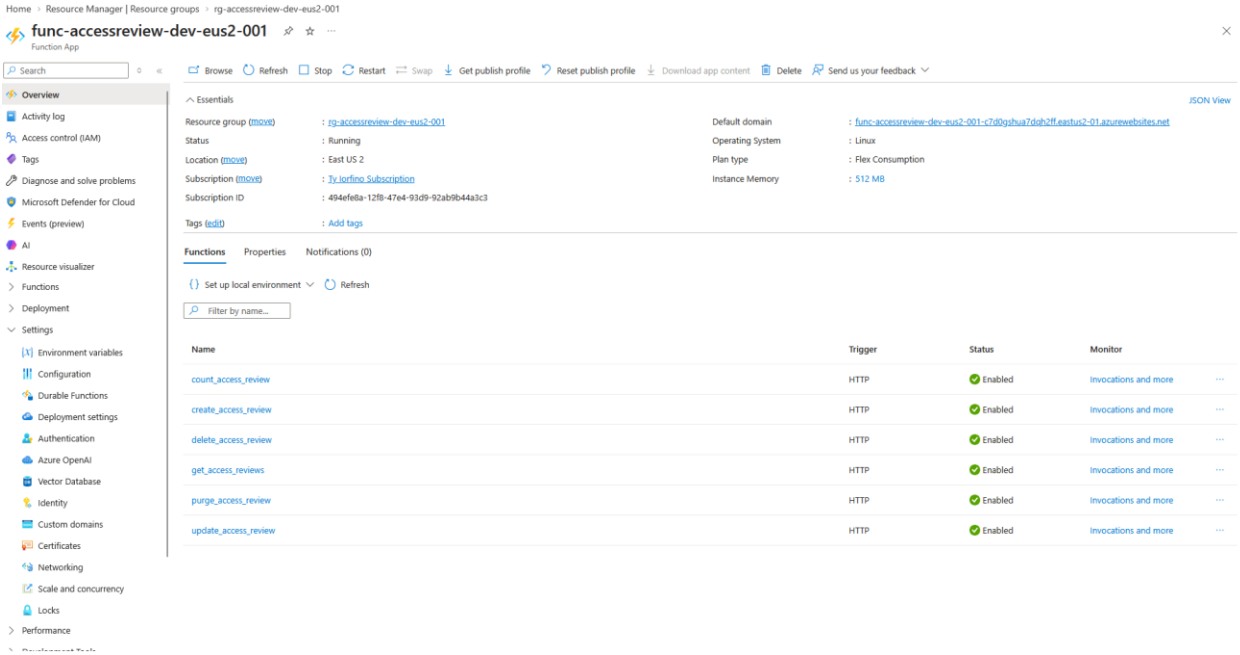


Figure 1: Azure Function App Overview Page

3.3 Resource Group

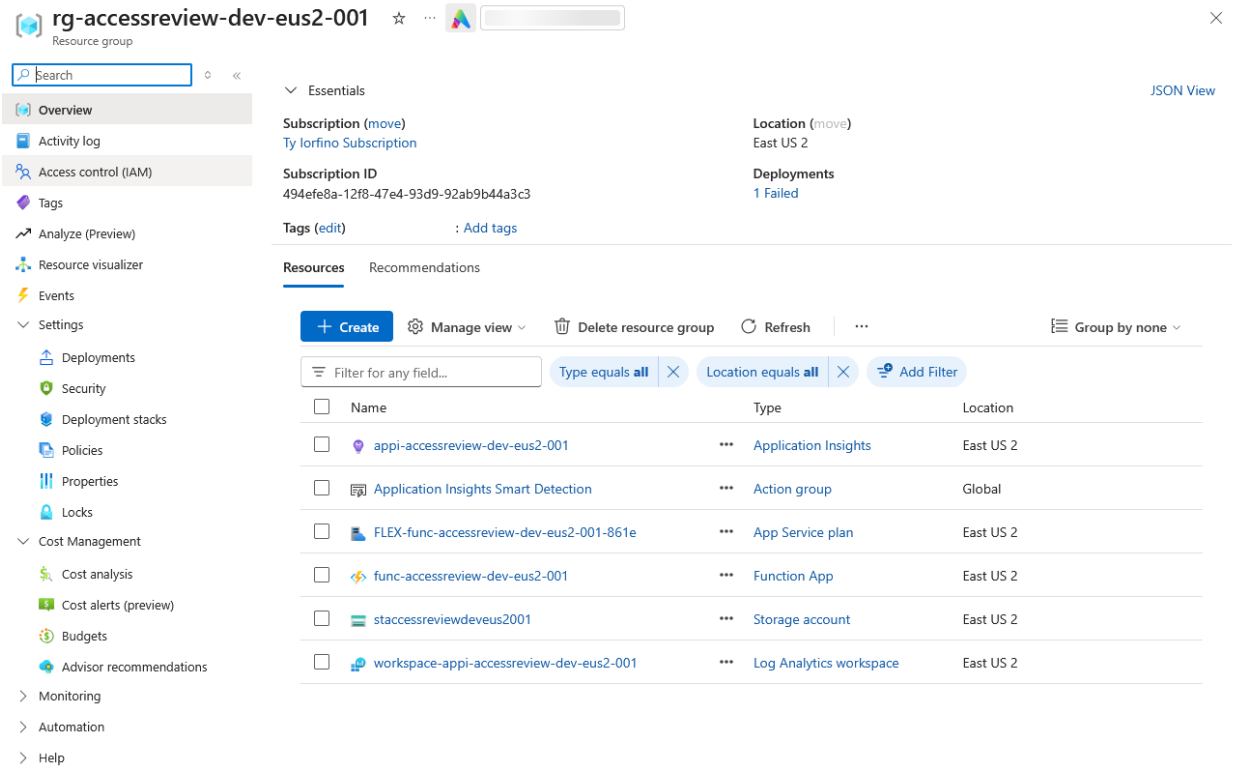


Figure 2: Complete List of Resource Groups

3.4 API Key Configuration

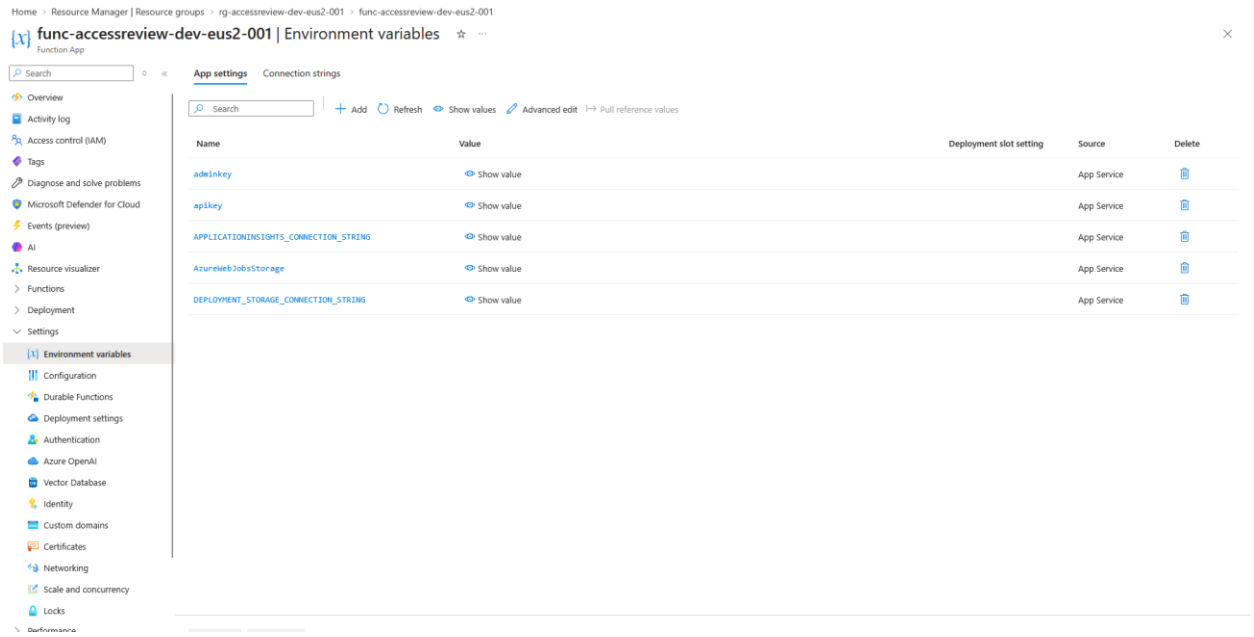


Figure 3: Environment Variables API Key Configurations

3.5 Successful Request on Azure

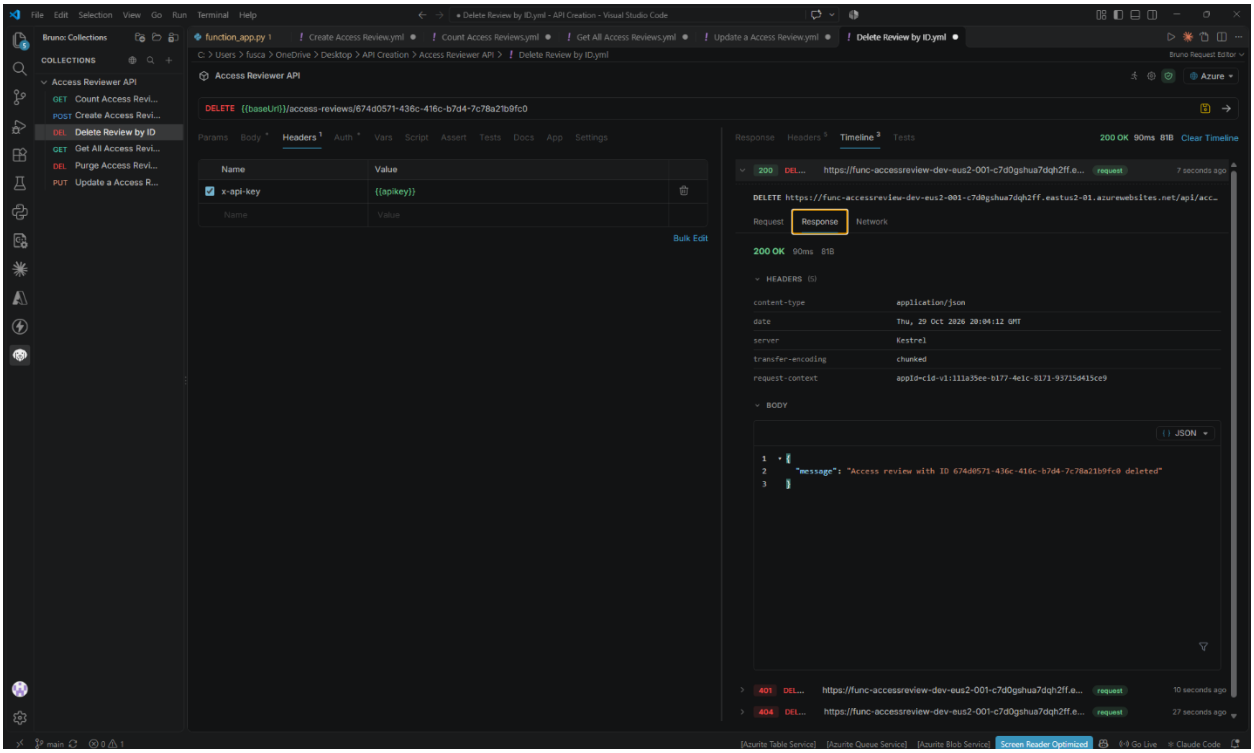


Figure 4: Successful Request in Azure Delete Request

3.6 Unauthorized Request on Azure

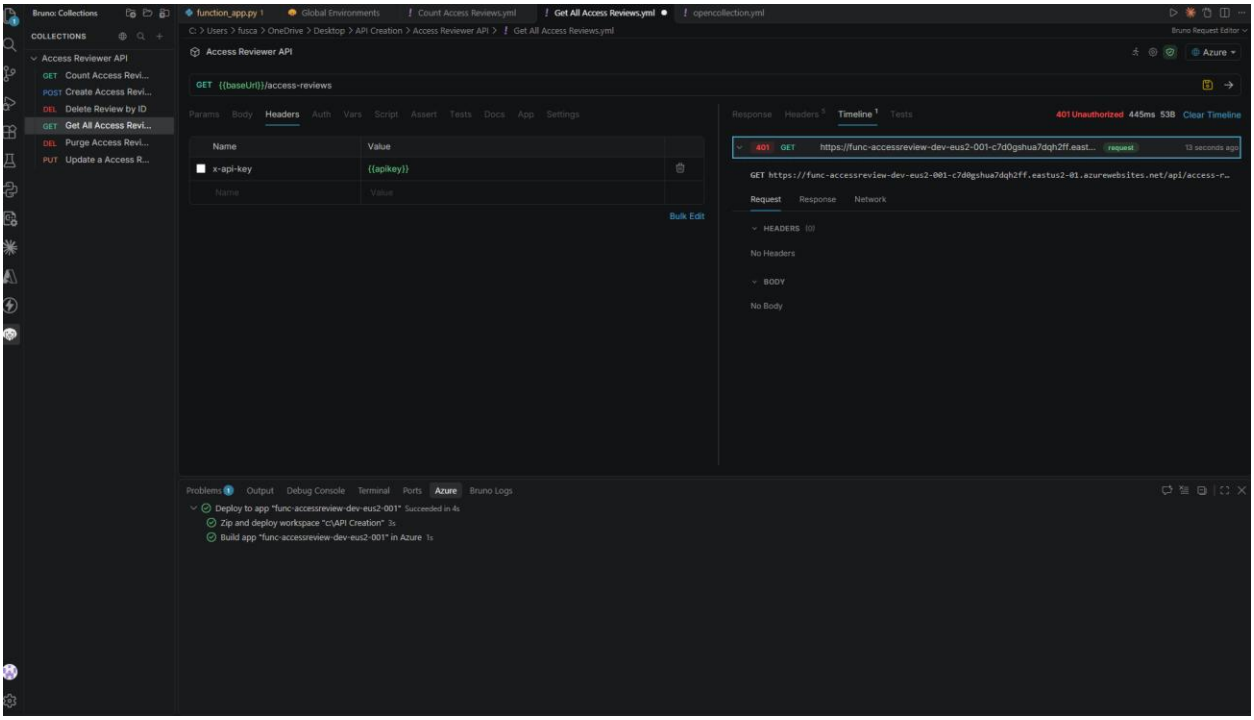


Figure 5: 401 Error in GET request due to no auth header

4. Testing Evidence

All tests below were run against the Azure base URL using Bruno, in the order listed, after the endpoints were first tested locally. An error test that returns the expected error code is a PASS. The purge tests send both the x-api-key and x-admin-key headers unless the test says otherwise.

4.1 Test Summary

#	Test Scenario	Endpoint / Method	Result
1	Create record A (PIM)	`POST /api/access-reviews`	PASS
2	Create record B (PAW)	`POST /api/access-reviews`	PASS
3	Get all access reviews	`GET /api/access-reviews`	PASS
4	Count access reviews	`GET /api/access-reviews/count`	PASS
5	Update record A (revoke access)	`PUT /api/access-reviews/{id}`	PASS
6	Create with no request body	`POST /api/access-reviews`	PASS
7	Update with no request body	`PUT /api/access-reviews/{id}`	PASS
8	Get all with no API key	`GET /api/access-reviews`	PASS
9	Get all with wrong API key	`GET /api/access-reviews`	PASS
10	Update an id that doesn't exist	`PUT /api/access-reviews/fake-id`	PASS
11	Delete record B	`DELETE /api/access-reviews/{id}`	PASS
12	Delete record B again	`DELETE /api/access-reviews/{id}`	PASS
13	Purge without admin key	`DELETE /api/access-reviews`	PASS
14	Purge with API key and admin key	`DELETE /api/access-reviews`	PASS
15	Count after purge	`GET /api/access-reviews/count`	PASS

4.2 Test Records

Test 1: Create record A (PIM)

Field	Details
Test Scenario	Create record A (PIM)
Endpoint / HTTP Method	`POST /api/access-reviews`
Expected Outcome	201 Created, record A returned with a new server-generated id
Actual Outcome	201 Created. Record A was returned with a new server-generated id (0fc0390c-5d6c-44ca-ac00-26c73defd00d).
Result	PASS
Notes	Request body shown on the left

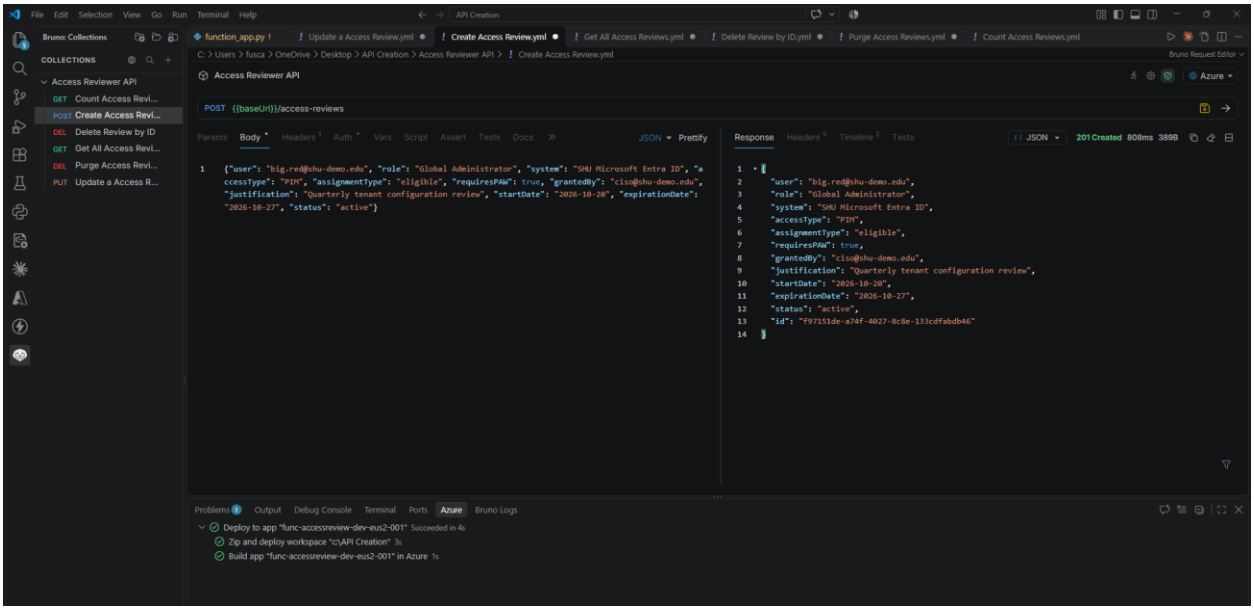


Figure 6: Create record A returns 201 created

Test 2: Create record B (PAW)

Field	Details
Test Scenario	Create record B (PAW)
Endpoint / HTTP Method	`POST /api/access-reviews`
Expected Outcome	201 Created, record B returned with a new server-generated id
Actual Outcome	201 Created. Record B was returned with a new server-generated id.

Field	Details
Result	PASS
Notes	Request shown on the left

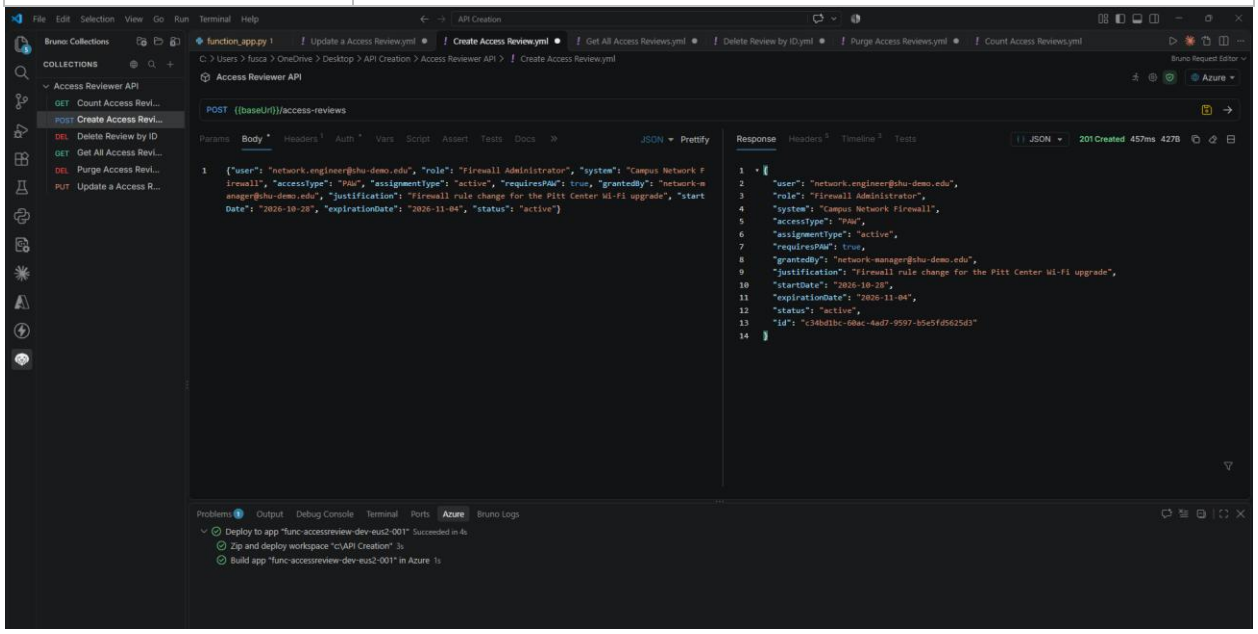


Figure 7: Create record B returns 201 created

Test 3: Get all access reviews

Field	Details
Test Scenario	Get all access reviews
Endpoint / HTTP Method	`GET /api/access-reviews`
Expected Outcome	200 OK, list containing records A and B
Actual Outcome	200 OK. The list returned records A and B with their ids.
Result	PASS
Notes	No additional notes

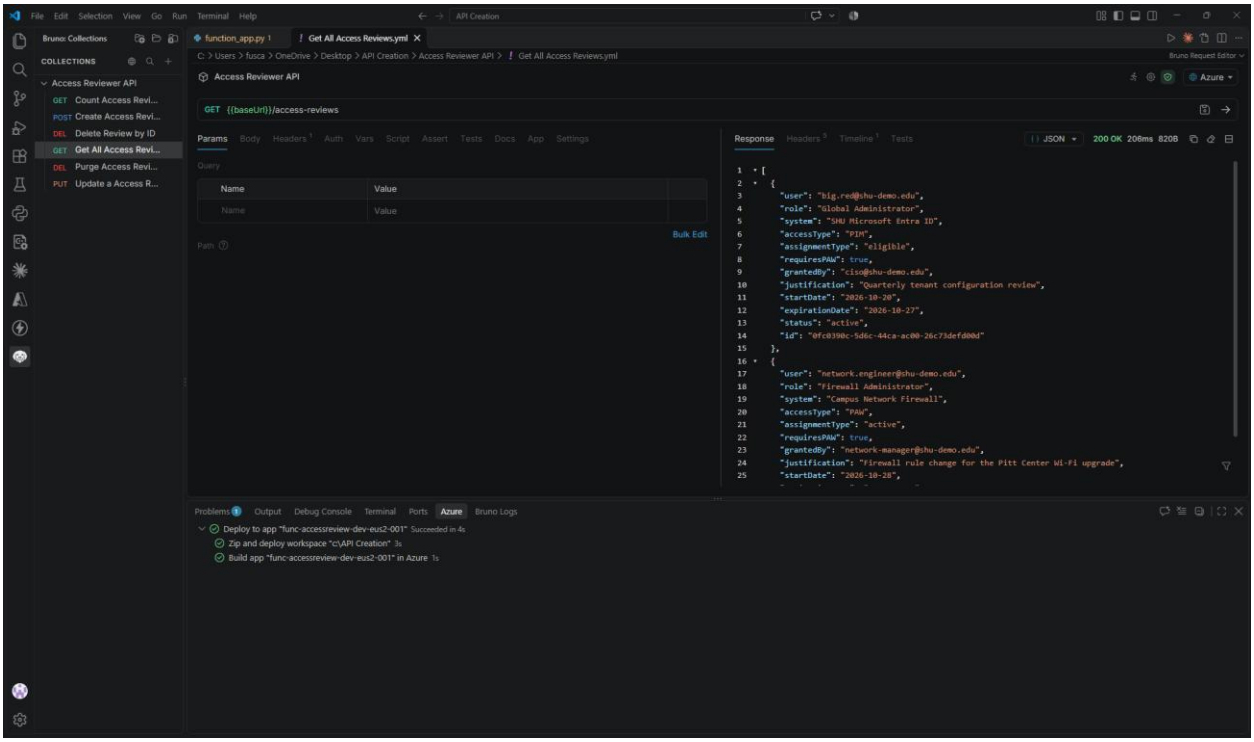


Figure 8: Get all access reviews returns 200 OK with both records

Test 4: Count access reviews

Field	Details
Test Scenario	Count access reviews
Endpoint / HTTP Method	`GET /api/access-reviews/count`
Expected Outcome	200 OK, {"count": 2}
Actual Outcome	200 OK, {"count": 2}
Result	PASS
Notes	Count matches the total number of records created in the tests

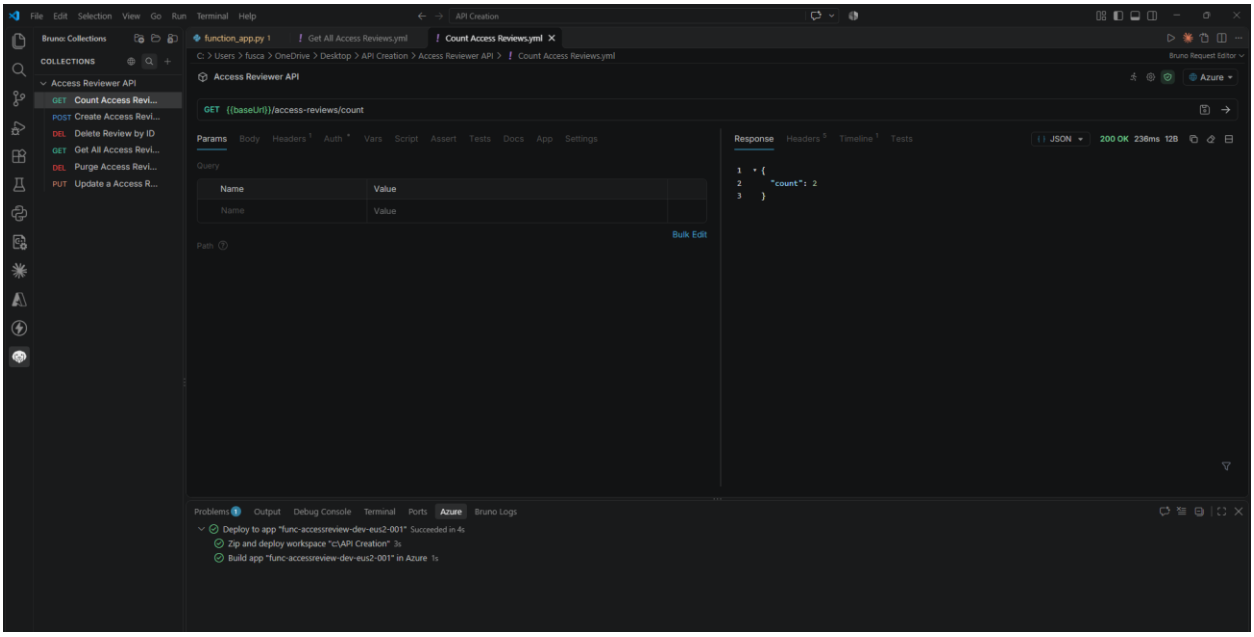


Figure 9: Count access reviews returns 200 OK with a count of 2

Test 5: Update record A (revoke access)

Field	Details
Test Scenario	Update record A (revoke access)
Endpoint / HTTP Method	`PUT /api/access-reviews/{id}`
Expected Outcome	200 OK, record A returned with "status": "revoked" and the same id
Actual Outcome	200 OK. Record A was returned with "status": "revoked" and the same id.
Result	PASS
Notes	Users record A id is used in the URL and isn't changed after the update

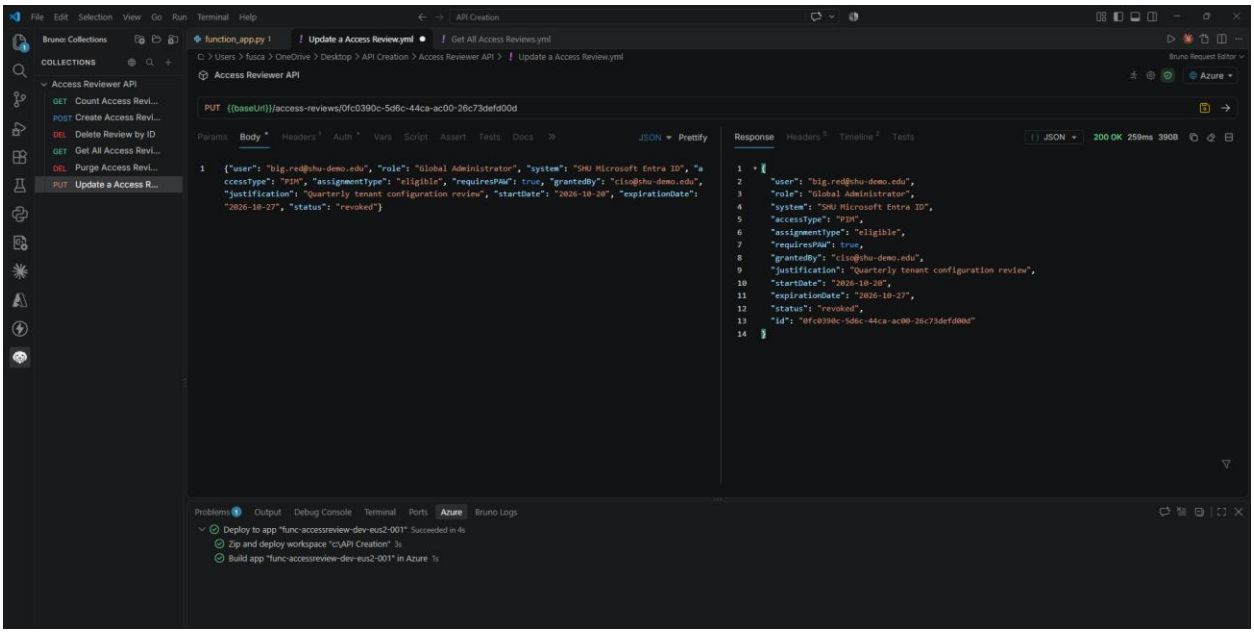


Figure 10: Update record A returns 200 OK with status revoked

Test 6: Create with no request body

Field	Details
Test Scenario	Create with no request body
Endpoint / HTTP Method	`POST /api/access-reviews`
Expected Outcome	400 Bad Request with an error message
Actual Outcome	400 Bad Request, {"error": "HTTP request does not contain valid JSON data"}
Result	PASS
Notes	There was no body sent in Bruno

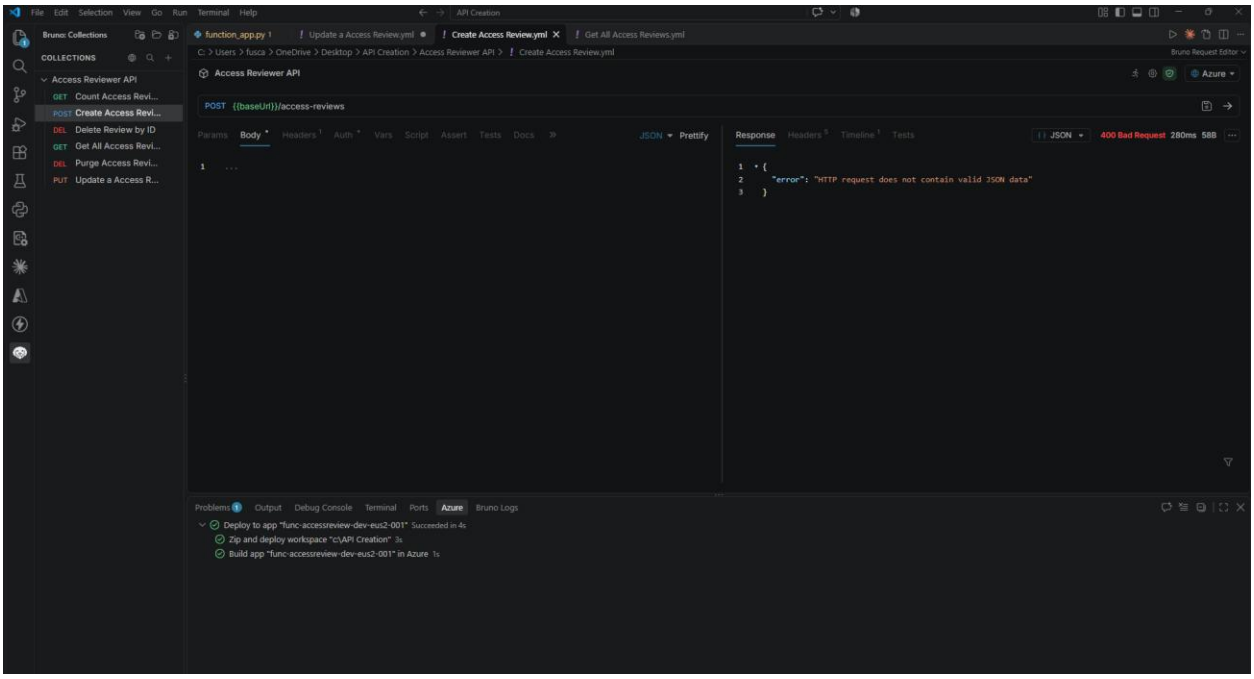


Figure 11: Create with no body returns 400 Bad Request

Test 7: Update with no request body

Field	Details
Test Scenario	Update with no request body
Endpoint / HTTP Method	`PUT /api/access-reviews/{id}`
Expected Outcome	400 Bad Request with an error message
Actual Outcome	400 Bad Request, {"error": "HTTP request does not contain valid JSON data"}
Result	PASS
Notes	Body set to no body in Bruno and sent to record A's id

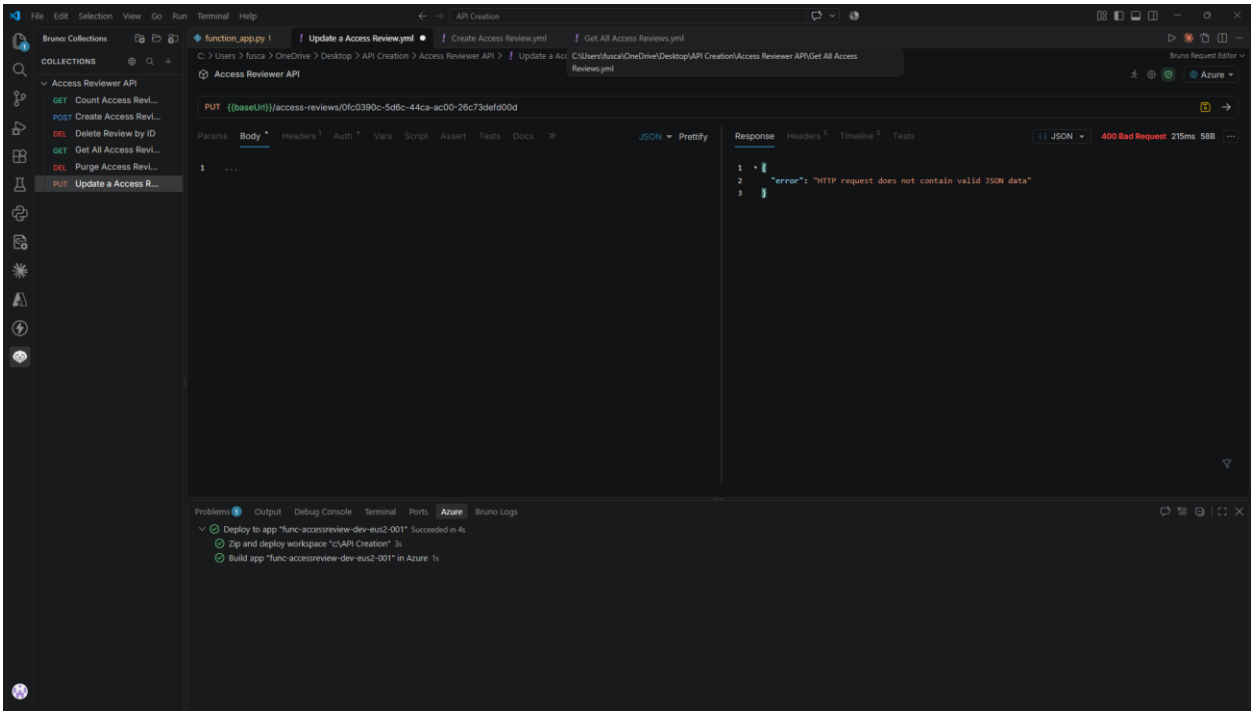


Figure 12: Update with no body returns 400 Bad Request

Test 8: Get all with no API key

Field	Details
Test Scenario	Get all with no API key
Endpoint / HTTP Method	`GET /api/access-reviews`
Expected Outcome	401 Unauthorized, {"error": "Unauthorized: Invalid or missing API key"}
Actual Outcome	401 Unauthorized, {"error": "Unauthorized: Invalid or missing API key"}
Result	PASS
Notes	The x-api-key header was unticked in Bruno

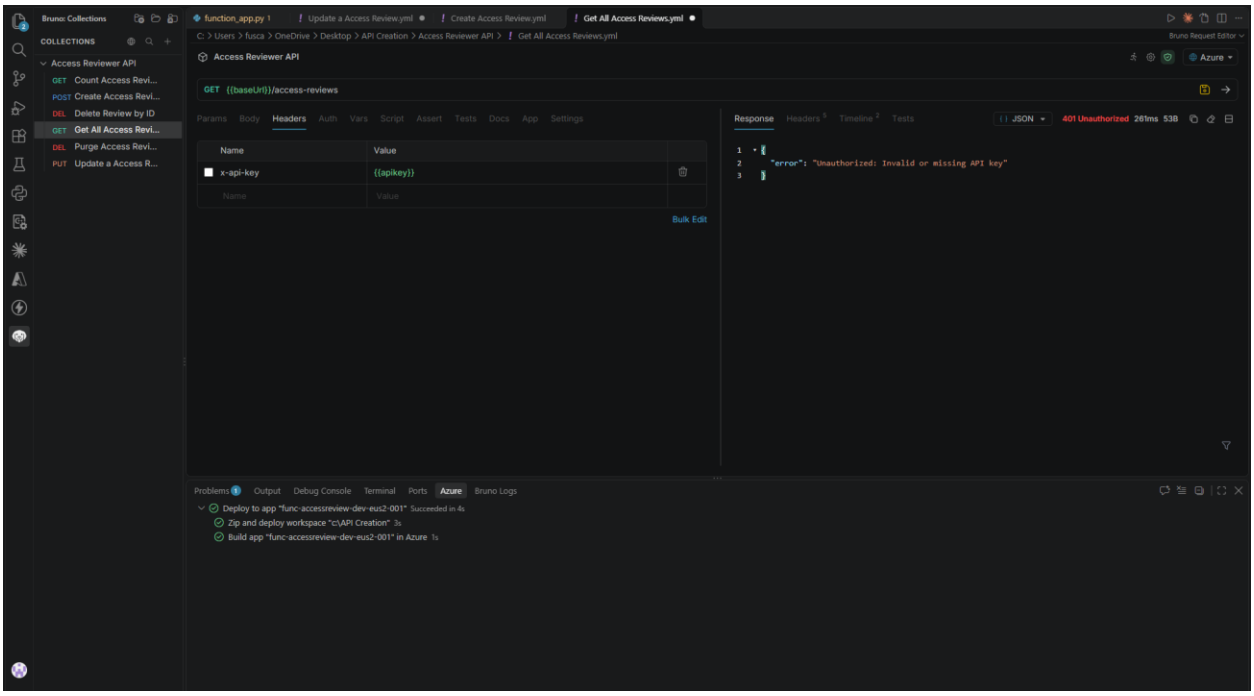


Figure 13: Get all with no API key returns 401 Unauthorized

Test 9: Get all with wrong API key

Field	Details
Test Scenario	Get all with wrong API key
Endpoint / HTTP Method	`GET /api/access-reviews`
Expected Outcome	401 Unauthorized, same error message
Actual Outcome	401 Unauthorized, {"error": "Unauthorized: Invalid or missing API key"}
Result	PASS
Notes	The x-api-key header was set to an invalid value instead of the real key

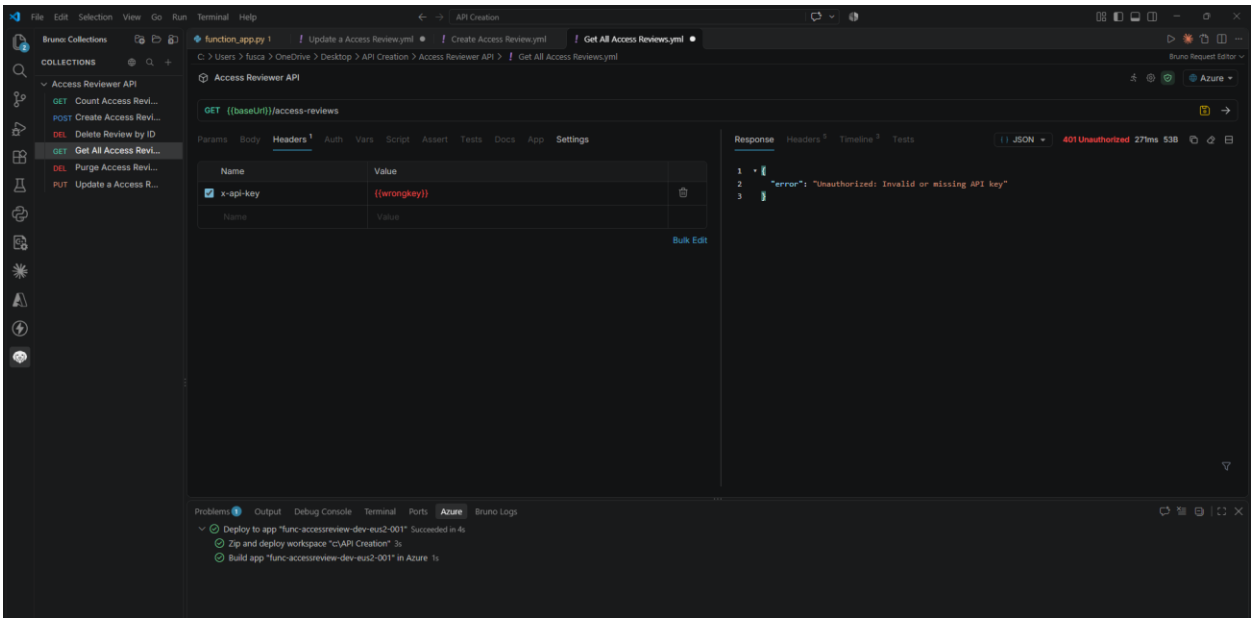


Figure 14: Get all with wrong API key returns 401 Unauthorized

Test 10: Update an id that doesn't exist

Field	Details
Test Scenario	Update an id that doesn't exist
Endpoint / HTTP Method	`PUT /api/access-reviews/fake-id`
Expected Outcome	404 Not Found, {"error": "Access review 'fake-id' not found"}
Actual Outcome	404 Not Found, {"error": "Access review with ID 0fc0390c-5d6c-44ca-ac00-26c73defd00 not found"}
Result	PASS
Notes	Used a non-existent id

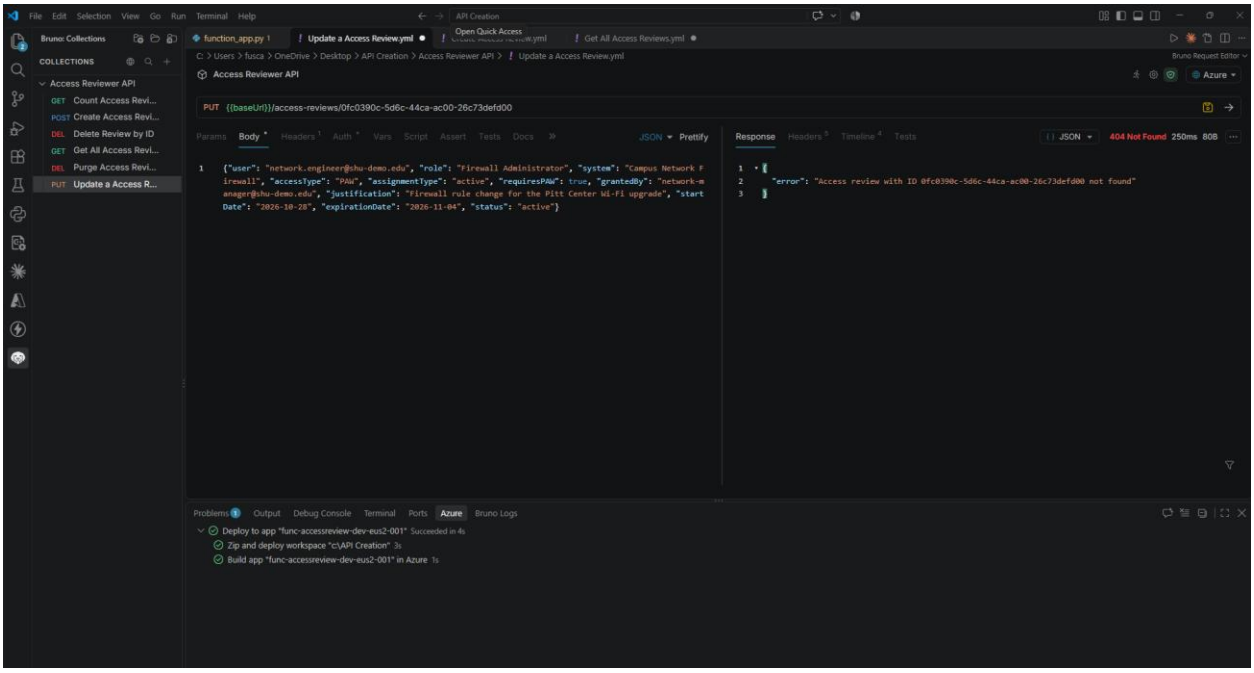


Figure 15: Update a non-existent id returns 404 Not Found

Test 11: Delete record B

Field	Details
Test Scenario	Delete record B
Endpoint / HTTP Method	`DELETE /api/access-reviews/{id}`
Expected Outcome	200 OK, {"message": "Access review deleted"}
Actual Outcome	200 OK, {"message": "Access review with ID 85e96e04-378e-4043-bf4a-d4491c037bd5 deleted"}
Result	PASS
Notes	Deleted record B using its id from test 2

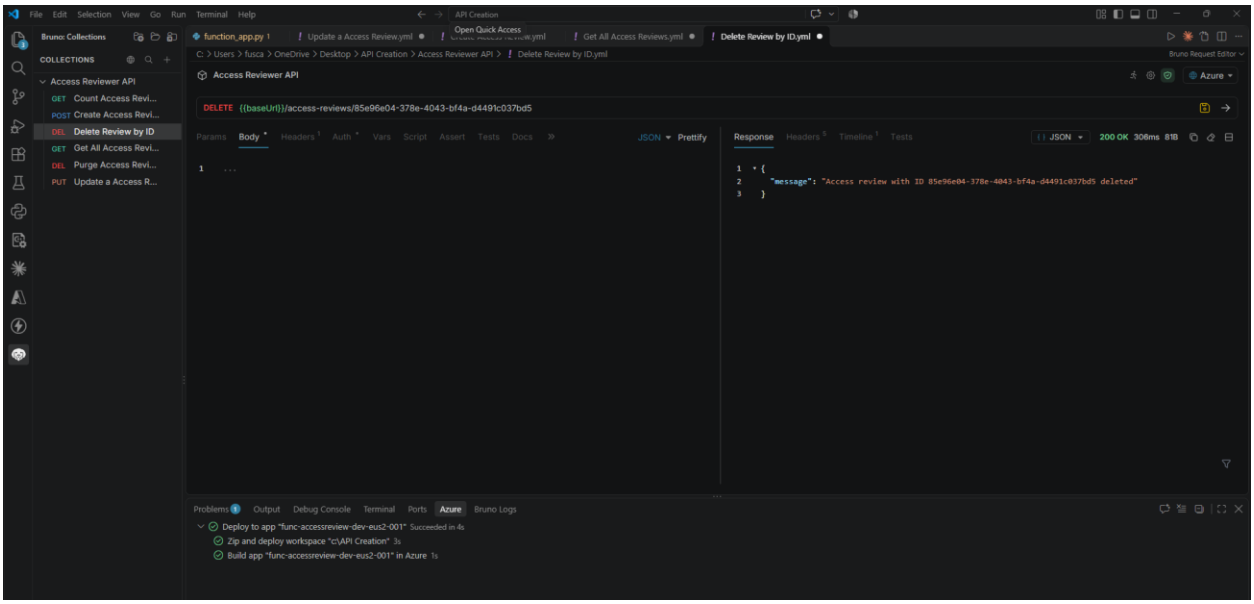


Figure 16: Delete record B returns 200 OK

Test 12: Delete record B again

Field	Details
Test Scenario	Delete record B again
Endpoint / HTTP Method	`DELETE /api/access-reviews/{id}`
Expected Outcome	404 Not Found, record B no longer exists
Actual Outcome	404 Not Found, {"error": "Access review with ID 85e96e04-378e-4043-bf4a-d4491c037bd5 not found"}
Result	PASS
Notes	Sent the same DELETE request as the last test but the record B no longer exists

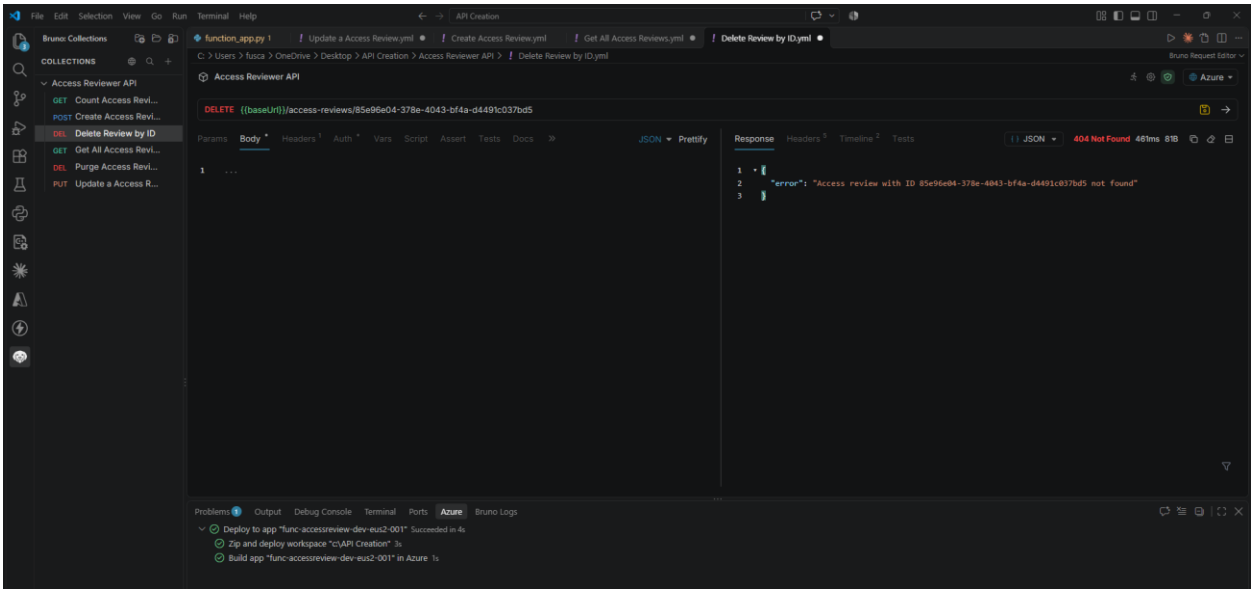


Figure 17: Delete record B again returns 404 Not Found

Test 13: Purge without admin key

Field	Details
Test Scenario	Purge without admin key
Endpoint / HTTP Method	`DELETE /api/access-reviews`
Expected Outcome	401 Unauthorized, {"error": "Unauthorized: Admin key required for purge"}; nothing deleted
Actual Outcome	401 Unauthorized, {"error": "Unauthorized: Admin key required for purge"}
Result	PASS
Notes	Only the x-api-key header was sent not the admin key. No records were deleted

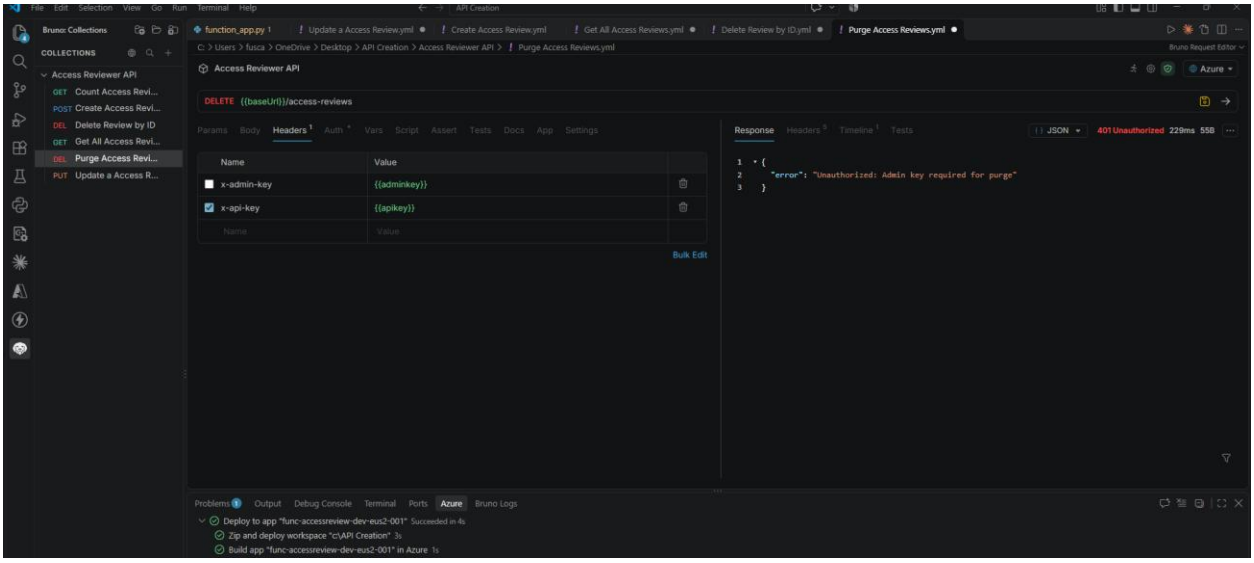


Figure 18: Purge without admin key returns 401 Unauthorized

Test 14: Purge with API key and admin key

Field	Details
Test Scenario	Purge with API key and admin key
Endpoint / HTTP Method	`DELETE /api/access-reviews`
Expected Outcome	200 OK, {"message": "All access reviews deleted", "deleted": 1}
Actual Outcome	200 OK, {"message": "All access reviews deleted. Total deleted: 1"}
Result	PASS
Notes	Both x-api-key and x-admin-key were sent, all records were purged

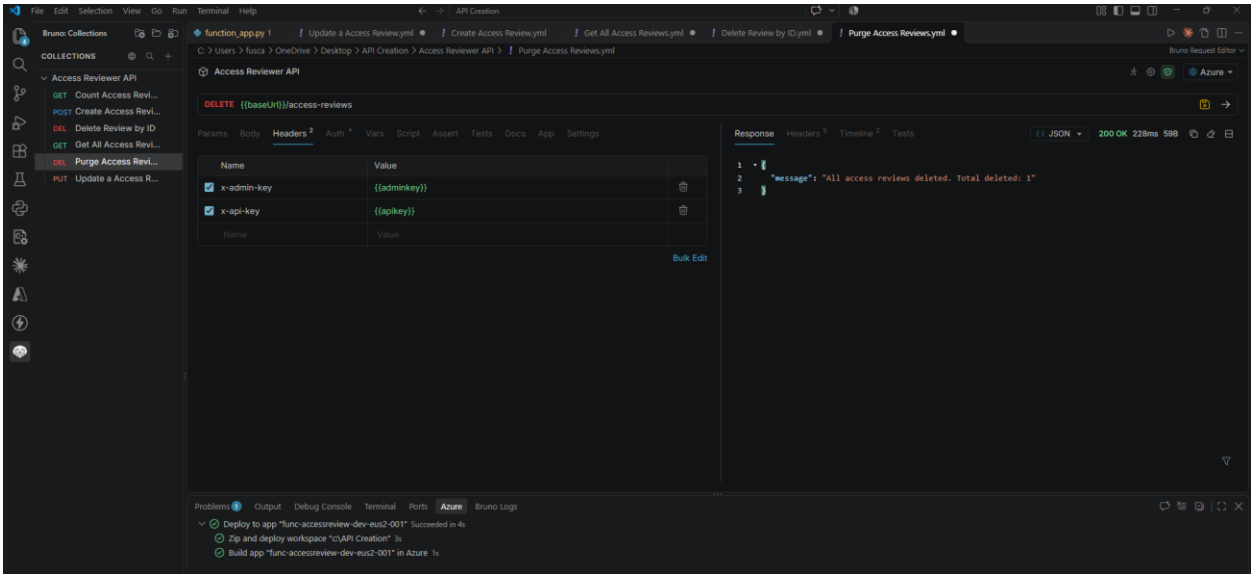


Figure 19: Purge with body keys return 200 OK

Test 15: Count after purge

Field	Details
Test Scenario	Count after purge
Endpoint / HTTP Method	`GET /api/access-reviews/count`
Expected Outcome	200 OK, {"count": 0}
Actual Outcome	200 OK, {"count": 0}
Result	PASS
Notes	Confirms the repository is empty after the purge

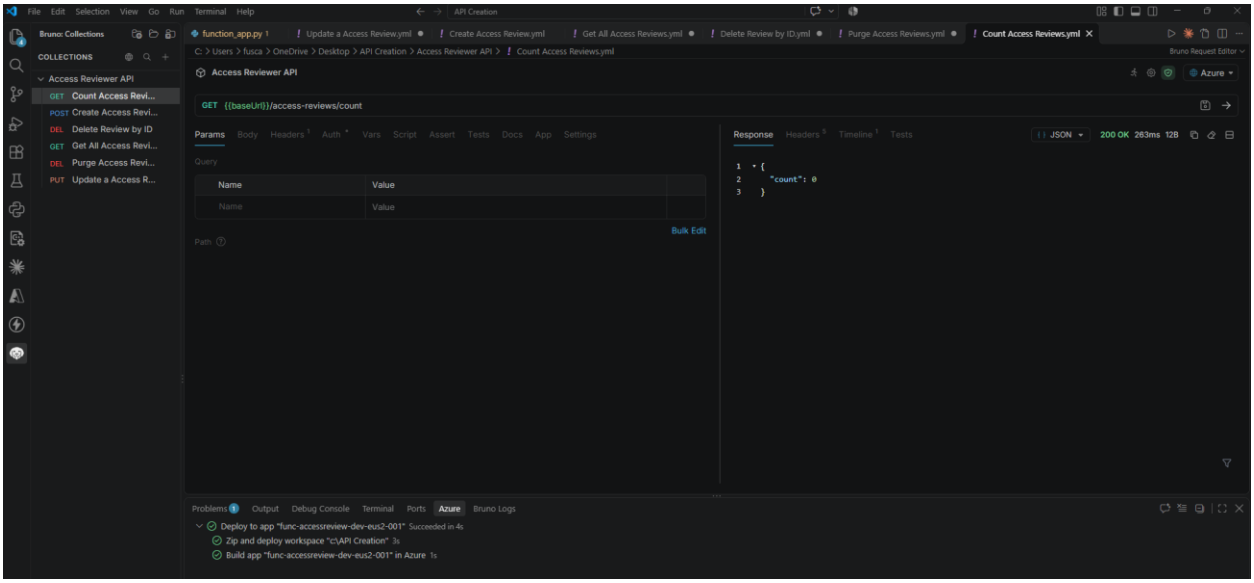


Figure 20: Count after purge returns 200 Ok with a count of 0

5. Conclusion

For this midterm I built the Access-Review-Delegation-Tracker API, a cloud-native REST API that tracks privileged and standard access delegations for PIM, PAM, PAW and Standard roles. It was written in Python with the Azure Functions V2 programming model and provides full CRUD on an in-memory repository, plus count and purge enhancements. Every endpoint is protected by an API key, bulk deletion also requires an admin key, and errors are handled consistently through a single exception wrapper. The API was tested locally, deployed to an Azure Function App on the Flex Consumption plan, and verified against the Azure URL with 15 tests that all passed.

This project taught me how serverless functions are structured, routed and deployed, how to keep secrets out of source code with environment variables, and how cloud hosting behaves differently from local development. The biggest lesson was that in-memory data is not shared or durable once Azure scales out or recycles instances. Next, I would add persistent storage, input validation for each field, endpoints to get a single record and filter by access type, and Azure Key Vault to store the API key and admin key securely, with the Function App reading them through Key Vault references instead of plain environment variables.